

Project 3: Training CNN with ADMM

Group No. 7 Member: Yanyun LIU

Introduction

In this project, I follow the instructions from two references[1,2] and a GitHub ADMM project [3] to implement a CNN model using ADMM algorithm.

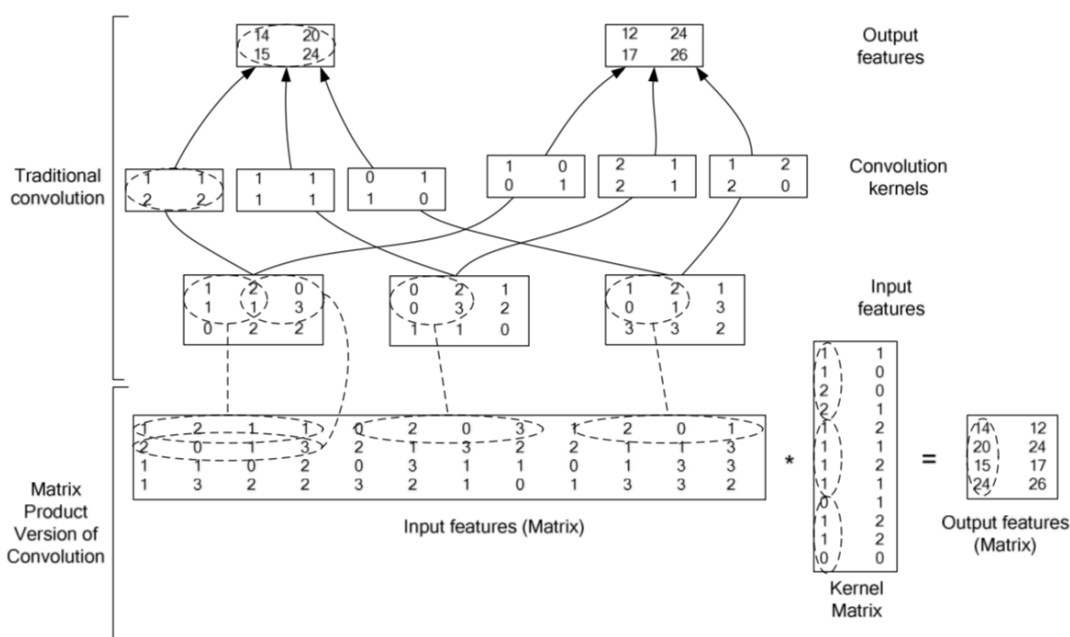
Implement Detail

1. Convert convolution to matrix multiplication: im2col

Since ADMM algorithm requires matrix computation, the convolution operation should be converted as matrix multiplication. The following figure[2] illustrates how im2col works.

The idea of im2col is shown below:

- Extract the local region in input image where will perform convolution later;
- Stretched it out into column;
- Concatenate the stretched columns into a large matrix;
- Flatten and concatenate kernel weight matrices into another matrix;
- Perform matrix multiplication between these two matrices.



The code implement im2col is shown below:

```

def im2col_2d(input_layer, kernel_size):

    field_width, field_lenght, field_channel = input_layer.shape
    #padding
    input_padding = np.zeros((field_width+2, field_lenght+2,
field_channel))
    input_padding[1:field_width+1,1:field_lenght+1,:] = input_layer

    input_layer = input_padding
    field_width, field_lenght, field_channel = input_layer.shape

    resize_list = []

    for i in range(0,(field_lenght-kernel_size+1)):
        for j in range(0,(field_width-kernel_size+1)):
            feature_row = []

            for k in range(0, field_channel):

                feature_row.append(input_layer[i:i+kernel_size,j:j+kernel_size,k])
                feature_row = np.concatenate(feature_row)

            feature_row = feature_row.reshape(1,
(kernel_size**2)*field_channel)
            resize_list.append(feature_row)

    return np.transpose(np.concatenate(resize_list).reshape((field_lenght-
kernel_size+1)**2, (kernel_size**2)*field_channel))

```

2. Convert im2col back to original image: col2im

Col2im is the inverse of im2col, which means we need to transfer the matrix back to original image, so we can perform pooling and other operation later.

3. ADMM Algorithm

In this project, I use ADMM algorithm to implement a 2 conv-layers CNN model with mnist dataset. For simplicity, I reduce the multi-class classification problem into a binary classification task. I extract the digit with label 0 and 8, and I will only give a bianry classification for these two digits. For the ADMM code structure, see reference[3].

The workflow for ADMM algorithm is shown below:

Step 1: Initialize all the weights:

```

layer_1_feature_num = 10
layer_2_feature_num = 10
layer_3_feature_num = 128
layer_4_feature_num = 128
layer_5_feature_num = 1

kernel_size = 3

input_size = 28
input_size_pool1 = int(input_size/2)
input_size_pool2 = int(input_size_pool1 / 2)

W_1 = np.zeros((layer_1_feature_num, kernel_size ** 2))
W_2 = np.zeros((layer_2_feature_num, layer_1_feature_num * 9))
W_3 = np.zeros((layer_3_feature_num, input_size_pool2 *
input_size_pool2 * layer_1_feature_num))
W_4 = np.zeros((layer_3_feature_num, layer_4_feature_num))
W_5 = np.zeros((layer_5_feature_num, layer_4_feature_num))

z_1 = np.random.randn(layer_1_feature_num, input_size**2)
z_2 = np.random.randn(layer_2_feature_num,
input_size_pool1*input_size_pool1)
z_3 = np.random.randn(layer_3_feature_num, 1)

```

Step 2: For the first convolution layer:

update W_1

```

a_0 = im2col_2d(a_0, kernel_size)
a_0_pinv = np.linalg.pinv(a_0)
W_1 = np.matmul(z_1, a_0_pinv)

```

For a_1, compute the left part first:

```

a_1_left = np.linalg.inv((beta * np.matmul(np.transpose(W_2), W_2) + gamma
* np.eye(layer_1_feature_num*kernel_size*kernel_size, dtype=float)))

```

and the right part. I apply col2im before computing the right-hand part.

```

z_1_change = col2im_2d(np.transpose(z_1),input_size,
kernel_size,layer_1_feature_num)
z_1_change = max_pooling(z_1_change, 2)
z_1_change = z_1_change.reshape(input_size_pool1**2,layer_1_feature_num)
z_1_change = np.transpose(np.repeat(z_1_change,9, axis=1))

a_1_right = (beta * np.dot(np.transpose(W_2), z_2) + gamma *
relu_layer(z_1_change))
a_1 = np.matmul(a_1_left,a_1_right)

```

and update z, according to the reference paper[1][3], min z can be determined by a if-else condition since active function is non-linear:

```

find_min = np.vectorize(get_z_1)
z_1 = find_min(a_1, z_1_change)

# for function get_z_1, see reference[3]
def get_z_1(a,w_a):

    def f_z(z):
        return gamma*(a-activation(z))**2 + beta*(z-w_a)**2

    z1 = max((a*gamma+w_a*beta)/(beta+gamma),0)
    result1 = f_z(z1)

    z2 = min(w_a,0)
    result2 = f_z(z2)

    if result1 <= result2:
        return z1
    else:
        return z2

```

and for the rest convolution layer, the step is nearly the same except the dimension.

Step 3: For fully connected layer, follow the step in [1]. You can find more details in the submitted code.

Step 4: Update parameters until convergence.

Step 5: Give prediction.

```

def get_predict_label(test, input_size):
    # test = np.random.randn(28, 28)
    # test = test.reshape(28, 28, 1)
    test = im2col_2d(test, 3)

    test = np.matmul(np.transpose(test), np.transpose(W_1))

```

```

    test = col2im_2d(test, input_size, kernel_size=3,
feat_num=layer_1_feature_num)
    test = relu_layer(test)

    test = max_pooling(test, stride_num=2)

    test = im2col_2d(test, 3)
    test = np.matmul(np.transpose(test), np.transpose(W_2))

    test = col2im_2d(test, int(input_size / 2), kernel_size=3,
feat_num=layer_2_feature_num)
    test = relu_layer(test)

    test = max_pooling(test, stride_num=2)

    print(test.flatten().shape)
    test = test.flatten()

    test = np.matmul(np.transpose(test), np.transpose(W_3))
    test = relu_layer(test)

    test = np.matmul(np.transpose(test), np.transpose(W_4))
    test = relu_layer(test)

    test = np.matmul(np.transpose(test), np.transpose(W_5))
    test = relu_layer(test)
    print(test)

    pre = vget_predict(test)

    print(pre)

    return pre

```

and I have my own pooling and Relu function shown below:

```
def max_pooling(input_layer, stride_num = 2):
    row, col, features = input_layer.shape

    pool_row = row // stride_num
    pool_col = col // stride_num

    return input_layer[:pool_row * stride_num, :pool_col *
stride_num,:].reshape(pool_row, stride_num, pool_col, stride_num,
features).max(axis=(1, 3))

def relu_layer(input_layer):
    input_layer[input_layer < 0] = 0
    return input_layer
```

Result and Comparison

Since I didn't use cython to accelerate training stage, so the updating part is time consuming. I do all the test within one epoch.

The ADMM algorithm can reach about 69% accuracy after one epoch, and I use a same structure CNN without drop-out and batchnormalization, after first epoch, the training accuracy is 68%, not worse than the traditional CNN architectures.

Reference

- [1]. Taylor G, Burmeister R, Xu Z, et al. Training neural networks without gradients: A scalable admm approach[C]//International Conference on Machine Learning. 2016: 2722-2731.
- [2].Chellapilla K, Puri S, Simard P. High performance convolutional neural networks for document processing[C]//Tenth International Workshop on Frontiers in Handwriting Recognition. Suvisoft, 2006.
- [3].https://github.com/dongzhuoyao/admm_nn